

RECUT: Complete Prototype and Validation

Recovering an LLM inference window with less repeated work

Stage 3 research artifact | 16 September 2026

Prepared for Nazish Baliyan and discussion with Prof. Abdulrahman Mahmoud. This report completes a bounded implementation and evaluation stage. It is not a completed thesis, deployment certification or conference-acceptance prediction. Earlier RECUT studies remain unchanged and are not pooled into these results.

Measured outcome

The expanded prototype completed the prescribed three-model experiment and passed its correctness gates. Sparse replay reduced complete affected-window time for late-layer faults: full/sparse ratios were approximately 1.38-1.53x across the six model/shape strata. That benefit is conditional, not a general inference speedup. On clean sessions, sparse protection added about 5-9% over the bare engine at initial prefix 128, and 14-49% at prefix 2,048. Early-layer faults had no consistent benefit. The larger-model study therefore strengthens both the mechanism evidence and the warning: checkpointing, detection and buffered output remain costly. This is a completed bounded prototype/evaluation artifact, not an established A/A* contribution.

Main-study evidence	Completed result
Models / workloads	3 models, 18 model-workload combinations
Measured sessions	2,358; warmups and profiles excluded
Window outcomes	6,966 exact-checked commits; 54 specified safe refusals
State archives	21 main archives; 2,338 recovered/reference tensor pairs rechecked on CPU

What the project actually does

An autoregressive language model keeps a memory of previous tokens, called the KV cache. If part of that memory is corrupted, later internal states can become wrong too. Merely fixing the original damaged value does not undo those consequences. Our protected full-replay baseline repeats a whole uncommitted window from a trusted checkpoint. RECUT saves a few intermediate layer inputs, detects changed old-prefix cache layers, and tries to repeat only the necessary upper-layer work.

The important safety rule is about history: reuse is permitted only while the recovered token history matches the speculative history. After the first corrected token differs, every later step is recomputed from the first layer, even if token IDs later match again. Tokens remain private until the complete window is accepted.

What was completed in this stage

The controller now has canonical SHA256-preserving batched transfers, full/sparse/all-cuts policies, complete-window cost measurement, native and bare clean baselines, a larger 7B model, 2,048-token contexts, K/V fault strata, integrity challenges, recorded output holdback, and independently reloadable full-state snapshots. Debugging, main execution, independent checks and the delivery artifact are distinct evidence layers.

Completing the experiment means accounting for every prespecified outcome and reporting unfavorable findings too. It does not mean that the mechanism is novel, efficient enough for deployment, or ready for an A/A submission.*

1. Mechanism and conditional correctness

One protected window

Phase	Implemented behavior
Begin	Preserve the public committed state; make separate working and checkpoint copies; seal the checkpoint.
Speculate	Decode W steps privately. Sparse saves three layer-input cuts; all-cuts saves every nonzero layer input. Seal captured activations with owner, position, token and cut metadata.
Detect	Verify checkpoint integrity; compare each working old-prefix K/V tensor numerically against the checkpoint. The controller receives no injection coordinates.
Recover	Select the deepest retained cut no higher than the earliest changed layer. Replay from that cut while history agrees. A damaged required journal forces full replay.
Diverge	At the first corrected-token mismatch, truncate to the last valid processed position and permanently switch subsequent steps to full-layer replay.
Commit or refuse	Check finite state and checkpoint integrity again. Release the complete window or reject without adding output. A damaged checkpoint poisons the session.

Why selective reuse can be valid

A saved cut is the hidden input before that decoder layer executes. Under the declared fault model, layers below the earliest directly corrupted cache layer received no direct fault. While their complete token history agrees with recovery, their saved inputs are still valid. Rebuilding the affected upper layers then restores the clean output for that step. If the next-token decision changes, later lower-layer states no longer have a valid common history and cannot be reused. Full replay from that point restores the remaining trajectory.

This is a conditional code-grounded argument, not a mechanized proof or a minimum-recomputation theorem. The same argument extends across sequential committed windows only if state remains trustworthy between them.

The trust boundary is essential

The supported contract is persistent, finite, **numerically unequal** changes to old-prefix K/V during speculation after checkpoint capture. Every potentially corrupting layer must retain such evidence until detection. Initial state, model weights, host metadata/seal storage and guard/recovery/commit execution are trusted. There are no concurrent writers or faults during those trusted phases.

Signed-zero-only working-cache changes are invisible to numeric comparison; byte seals distinguish them in checkpoints/journals. Suffix-only, reverted, pre-checkpoint, between-window and computation-origin faults are outside the guarantee. A reverted low-layer fault combined with a surviving high-layer fault is also outside it: the higher alarm alone cannot justify reusing the damaged lower activation.

Checkpoint refusal preserves previous committed output; it does not repair that checkpoint or automatically restart the session. This is a sequential local API, not crash durability, network transactions or rollback of external actions. Clean-state equality does not make a model's answer truthful or safe.

2. Experimental design and measurement

Prespecified matrix

Dimension	Values and scope
Models	Pinned SmolLM2-135M, Qwen2.5-0.5B and Qwen2.5-7B base models; BF16; greedy decoding; one sequence; eager attention.
Workloads	Three authored synthetic passages: cloud, code and reasoning. Numbered repetitions form prefixes of 128 or 2,048 tokens; windows contain 8 or 16 steps respectively. Three consecutive windows per session.
Clean comparators	Native HF, bare custom engine, full/sparse/all-cuts batched, full/sparse legacy. Native/bare are unprotected cost references, not resilience-equivalent alternatives.
Fault treatments	Early, middle and late K/V changes; two staggered old-prefix faults; faults in every window. Finite fraction-bit-3 XOR or 8-RMS vector changes, fixed before measurement.
Guards	Required saved-journal corruption and checkpoint corruption. Journal treatment differs by policy and is excluded from fair paired timing.
Repetitions	Two fault seeds and two timing repetitions; three clean timing repetitions; separate warmups and synchronized profiles. Repetitions are not independent fault samples.
Denser-journal comparator	All nonzero layer cuts, under the same checkpoint/detector/commit contract. This is not a reproduction of HCache, HybridServe, LayerSkip or VeriCache.

All ordinary fault cases affect the second window; repeated-prefix faults affect every window. Strata couple layer, K/V, severity and injection time, so these factors cannot be given separate causal interpretations. Initial context and window length also change together between the two shapes. The inputs are not a representative accuracy or production-serving benchmark.

What the clocks include

The primary clock covers BEGIN through COMMIT/refusal: copies, seals, activation capture, speculative work, injection harness, detection, replay, final guards and synchronization. Session initialization is separately timed and added to initialization-inclusive session comparisons. Common input copying, native batched prefill and experimental native-reference checking remain excluded from both boundaries. Injection-subtracted costs are a diagnostic, not a deployment measurement.

Native/custom equality is checked at every post-prefill decode position from the same native batched-prefill checkpoint. It does not establish equivalence between different prefill execution shapes. Every committed window is compared with native-reference cache, logits, tokens and a two-step continuation.

For timing comparisons, first take the median of repetitions for each workload/scenario/seed/variant. Then form matched ratios and summarize across the stated cases. No population confidence interval, field fault rate or significance claim is inferred. A ratio above one favors the denominator; protected/unprotected cost ratios above one instead mean added protection cost.

Method order rotates, but three clean repetitions do not fully counterbalance seven variants. Small differences near 1x may reflect timing variation. Models can occupy different lab nodes of the same GPU type; across-model timing differences are not an isolated scaling law.

3. Normal-operation cost

Times below are milliseconds for a complete three-window clean session, including its separately measured initialization. Rows identify the initial prefix: 128 uses W=8; 2,048 uses W=16. Later windows have longer prefixes. Each entry is the median of three prompt-level case medians; each case first summarizes three timing repetitions.

Model / prefix	Native ms	Bare ms	Full ms	Sparse ms	All-cuts ms
Smol-135M / 128	748.9	749.2	790.9	795.5	827.8
Smol-135M / 2048	1482.8	1494.8	1816.6	1829.5	1885.4
Qwen-0.5B / 128	613.0	616.8	639.5	652.2	681.7
Qwen-0.5B / 2048	1219.0	1228.6	1386.6	1400.4	1460.0
Qwen-7B / 128	768.1	765.7	835.3	835.9	865.2
Qwen-7B / 2048	1592.8	1596.2	2371.5	2379.5	2427.9

Protected sparse cost relative to ordinary inference is shown separately. These are medians of three matched ratios, not ratios obtained by dividing the cross-prompt milliseconds displayed above.

Model / prefix	Sparse / bare	Sparse / native
Smol-135M / 128	1.06x	1.06x
Smol-135M / 2048	1.23x	1.23x
Qwen-0.5B / 128	1.05x	1.06x
Qwen-0.5B / 2048	1.14x	1.15x
Qwen-7B / 128	1.09x	1.09x
Qwen-7B / 2048	1.49x	1.50x

Normal-case protection is the main deployment concern. Qwen-7B at initial prefix 2,048 required about 2.38 seconds for its three protected sparse windows versus 1.59 seconds for native inference, with matched sparse/native cost about 1.50x. Both full and sparse protected methods pay much of that shared cost. Reducing replay after a fault cannot eliminate it. The short/long rows change window size together with prefix length, so they do not isolate a pure context-length effect.

Why this comparison matters

Recovery savings are paid only when a supported fault is detected. Copies, guards, capture and delayed release are paid on ordinary protected windows too. A speedup against protected full replay therefore does not imply faster ordinary inference. Both comparisons are necessary: a same-contract recovery comparison and an honest unprotected cost reference.

The execution path is a batch-one eager research engine. It does not include an optimized serving scheduler, concurrent requests, FlashAttention/SDPA, quantized caches or distributed execution. These numbers cannot be presented as production throughput, end-user TTFT or general model efficiency.

4. Recovery benefit and its limits

Full / sparse complete affected-window time. Above 1 favors sparse; below 1 favors full. Both use batched seals and the same protection contract. All eight prespecified fault strata are shown. Each ordinary stratum summarizes six matched case-window ratios (three prompts x two seeds); every-window faults summarize 18. Each ratio is formed after collapsing two timing repetitions. Guard challenges are excluded.

Initial prefix 128

Fault stratum	Smol-135M	Qwen-0.5B	Qwen-7B
Early K	0.99	1.00	1.00
Early V	0.99	0.99	1.00
Middle K	1.30	1.28	1.26
Middle V	1.32	1.29	1.27
Late K	1.53	1.52	1.46
Late V	1.51	1.50	1.46
Two-layer	1.11	1.13	1.11
Every-window	1.51	1.52	1.47

Initial prefix 2,048

Fault stratum	Smol-135M	Qwen-0.5B	Qwen-7B
Early K	0.99	0.99	1.00
Early V	1.00	0.99	0.99
Middle K	1.28	1.27	1.22
Middle V	1.26	1.28	1.21
Late K	1.46	1.49	1.38
Late V	1.46	1.50	1.38
Two-layer	1.10	1.12	1.10
Every-window	1.47	1.49	1.37

The expected shape appears: little or no benefit for early-layer corruption, moderate benefit for middle-layer corruption, and the largest benefit for late-layer corruption. Two-layer faults force a shallower valid cut and reduce savings. Qwen-7B's late-K/V ratios fall from about 1.46x at the short shape to about 1.38x at the long shape. These comparisons include the full affected-window protection path; they are not recovery-kernel-only timings or gains over unprotected inference.

Main-study replay windows with a changed corrected token / with subsequent partial-to-full replay: Smol-135M 12 / 4; Qwen-0.5B 0 / 0; Qwen-7B 0 / 0. The separate tiny pretrained debug matrix had 9 divergent replay windows and 4 partial-to-full windows. These are window executions including policies/repetitions, not independent faults. Forced-divergence regression tests also cover permanent fallback and later token reconvergence. All main token divergence occurred in two configured Smol fault cases. Only the code/multi-prefix case exercised partial-to-full recovery: four executions are two policies times two timing repetitions, not four independent faults. Its original main state was outside the predeclared snapshot subset. A separate, explicitly post-hoc replay archived that case and is described in Section 7. The debug matrix also contains a predeclared archived Qwen-0.5B fallback. No main Qwen token divergence was observed; correctness on one selected divergent case is not broad divergence-frequency evidence.

5. Whole-session costs and sensitivity

The following ratios include all three windows and session initialization. Each row summarizes 48 matched case-session ratios (three prompts x two seeds x eight fault strata), after two timing repetitions per case. This equal-case summary is not a deployment incidence mixture. All-cuts uses every nonzero layer cut; it is an internal denser-journal comparator, not a published-system reproduction.

Model / prefix	Full / sparse	Full / all-cuts	Sparse / all-cuts
Smol-135M / 128	1.13	1.08	0.97
Smol-135M / 2048	1.11	1.07	0.97
Qwen-0.5B / 128	1.12	1.08	0.96
Qwen-0.5B / 2048	1.11	1.08	0.97
Qwen-7B / 128	1.11	1.07	0.97
Qwen-7B / 2048	1.09	1.07	0.98

Across this equal-case fault mixture, sparse improved complete initialization-inclusive sessions over protected full replay by about 1.09-1.13x. All-cuts was generally slower than sparse despite finer cut placement; the two-layer stratum showed small case-level exceptions. Saving more activations is not automatically worthwhile. These internal comparisons do not establish that three fixed cuts are optimal or outperform adapted published serving/reconstruction systems.

Supported-incidence sensitivity

For matched complete windows, let H be sparse-clean cost minus full-clean cost and R be full-fault cost minus sparse-fault cost. Let p be a hypothetical fraction of windows affected by a supported fault. If H and R are both positive, sparse becomes cheaper in the illustrative mixture only above $p^* = H / (H + R)$. Other sign combinations must be reported separately. This compares two protected methods; it does not price their shared cost against unprotected inference or include one-time initialization.

Model / prefix	Defined / all	Median p^*	No extra H	Other signs
Smol-135M / 128	39/60	1.4%	13	8
Smol-135M / 2048	36/60	1.3%	16	8
Qwen-0.5B / 128	38/60	3.3%	13	9
Qwen-0.5B / 2048	48/60	2.6%	2	10
Qwen-7B / 128	39/60	1.7%	13	8
Qwen-7B / 2048	47/60	0.9%	6	7

Defined/all counts positive- H , positive- R thresholds out of all 60 affected case-windows per model/shape. The every-window stratum contributes three windows per case; the other seven strata contribute one each. Median p^* uses only that defined subset. 'No extra H' counts nonpositive H with positive R ; 'Other signs' retains all remaining cases. The defined-subset median thresholds range from roughly 0.9% to 3.3%, with important undefined/sign-changing cases retained in the table. This is a sensitivity calculation, not evidence that real faults occur that often. Small normal-case differences can change H 's sign; no uncertainty interval or statistically reliable free-protection claim is warranted. The substantial shared cost of either protected method relative to ordinary inference is absent from p^* and must still be justified.

The deliberately injected fault fraction is not a measured deployment incident rate. A favorable threshold does not establish that a real system reaches it, and an observed negative H may be a small timing difference rather than a dependable zero-cost result.

6. Sealing, memory and delayed output

The implemented optimization

Batched sealing packs canonical tensor bytes by device, makes fewer device-to-host transfers, and hashes the same ordered metadata/byte stream on the CPU. It preserves stage-2 SHA256 digests; it is not a new cryptographic method or GPU hashing kernel. The legacy ablation uses the same controller with per-tensor copies. Both modes skip irrelevant empty-journal hashes in full replay, so the ablation isolates batching rather than every earlier-stage change.

Legacy / batched initialization-inclusive session time. Above 1 favors batching. Clean columns summarize three matched prompt cases; late-V columns summarize six prompt/seed cases.

Model / prefix	Clean full	Clean sparse	Late-V full	Late-V sparse
Smol-135M / 128	1.02	1.01	1.01	1.01
Smol-135M / 2048	1.03	1.03	1.02	1.03
Qwen-0.5B / 128	1.01	1.01	1.01	1.00
Qwen-0.5B / 2048	1.03	1.03	1.04	1.03
Qwen-7B / 128	1.03	1.03	1.00	1.02
Qwen-7B / 2048	1.05	1.05	1.04	1.04

The byte-preserving batching implementation modestly improves measured cost. Qwen-7B clean-session legacy/batched ratios were about 1.03x at the short shape and 1.05x at the long shape. Results near 1.00x should not be oversold without a stronger timing study. Batching reduces transfer-call overhead but still stages and hashes cache bytes; it does not resolve the long-context protection cost or the need to hold outputs until commit.

6. Memory and delayed output (continued)

GPU allocation and journal payload

Peak GiB is the largest window allocation recorded across all measured variants/scenarios at that shape, not a whole-job startup/prefill peak. Journal KiB is a clean case-window payload median, not an allocator peak.

Model / prefix	Peak GiB	Sparse KiB	All-cuts KiB
Smol-135M / 128	0.32	27	261
Smol-135M / 2048	0.82	54	522
Qwen-0.5B / 128	0.98	42	322
Qwen-0.5B / 2048	1.25	84	644
Qwen-7B / 128	14.35	168	1512
Qwen-7B / 2048	15.60	336	3024

The next table shows the median of nine clean case-window cells after reducing timing repetitions. GPU increase is the window peak minus its starting allocated memory; it includes temporary copies/staging and harness effects, not just the journal or a production-only protection footprint. Checkpoint is logical sparse-policy payload. Host peak is unmeasured.

Model / prefix	Bare increase MiB	Sparse increase MiB	Checkpoint MiB
Smol-135M / 128	0.587	9.797	2.988
Smol-135M / 2048	5.932	137.625	45.352
Qwen-0.5B / 128	1.104	5.493	1.594
Qwen-0.5B / 2048	8.172	73.696	24.188
Qwen-7B / 128	2.780	24.517	7.438
Qwen-7B / 2048	31.768	344.199	112.875

First-token availability within clean windows

Milliseconds below are medians across nine prompt/window case cells after within-case repetition reduction. Protected sparse releases its window at commitment. Native/bare step-completion times are used as availability proxies; the harness still buffers those tokens locally. The final column is the median of each window's mean hold-after-step duration, not a pooled-token average or network latency.

Model / prefix	Native ms	Bare ms	Sparse ms	Median mean hold ms
Smol-135M / 128	31.3	31.1	261.6	117.6
Smol-135M / 2048	30.7	31.5	607.8	304.4
Qwen-0.5B / 128	26.0	25.8	215.8	95.9
Qwen-0.5B / 2048	25.6	25.8	464.8	228.7
Qwen-7B / 128	31.7	31.6	278.9	125.7
Qwen-7B / 2048	33.3	33.4	790.8	421.6

GPU allocator peaks include the resident model and native-reference fixtures; they are not a production minimum. Logical checkpoint/journal bytes omit allocator and protection overhead. Batched transfer staging adds temporary device and host storage; host peak memory was not measured. Seal-transfer counters do not count every transfer in the controller. Synchronized component profiles are separate diagnostics, not an exhaustive partition of unprofiled wall time.

7. Validation and evidence strength

All 7,020 main attempted windows are accounted for: 6,966 committed with recorded exact cache/logit/token and continuation agreement; 54 checkpoint challenges refused with the specified integrity reason and no additional output. There were 2,358 recovered window executions and 684 native/custom post-prefill parity positions. The 216 warmup and 72 profile sessions are excluded from those measured totals.

Commit totals include the clean native/bare reference baselines. Recovery counts include journal-integrity challenges; the paired performance tables exclude both guard strata.

Model	Audit checks	Tensor pairs	Partial-to-full
Smol-135M	312,645	854	4
Qwen-0.5B	313,628	686	0
Qwen-7B	392,720	798	0

Local audits passed 1,018,993 checks in total. The HPC audits check the same evidence; they are not counted as additional fault observations. The 21 main archives contain 2,338 actual/reference tensor pairs, all numerically equal; 2,338 also passed the stricter raw-byte comparison. Independent raw recomputation agreed on 81,309 compared report quantities.

The core/controller suite passed all 286 tests on each allocated GPU. The final local combined suite passed 296 tests and skipped 17 CUDA-only tests, including 27 separate offline reporting tests. Repeating a test suite on several GPUs does not multiply its number of distinct test cases.

What is independently checked

The separate auditor checks complete matrices, source/input/output hash bindings, fault placement and arithmetic, same-treatment pairing, route and divergence rules, commitment/refusal records, latency arithmetic and transfer accounting. It then reloads the predeclared tensor archives on CPU using restricted tensor/basic-data loading and compares recovered/native tensors. Other trials retain recorded live equality checks; they were not all independently re-executed from saved tensors.

A second calculation reconstructs the report's principal numerical results directly from raw sessions before comparing them with the analyzer. It does not treat summary.json as the numerical source of truth. These are internal AI-assisted checks, not human peer review or external replication.

Preserved reporting correction

The frozen analyzer originally displayed zero seal counters for refused windows: those counters are nested under rejection metadata. A separate reporting-only tool corrected those counter medians and grouped counter displays. Original summary.json and every raw record remain unchanged. Use summary_corrected.json with analysis_corrections.json, which lists the changed fields and binds all relevant hashes. No measured time, correctness outcome, performance ratio, source freeze or experimental integrity gate was changed.

7. Additional verification and limits (continued)

Separate post-hoc fallback validation

Smol's code/prefix-128/W8 multi-prefix case, seed 7431, repetition 0 exercised natural partial-to-full fallback but was outside the main predeclared snapshot subset. After observing that gap, a written amendment fixed one validation-only replay of the same workload, fault, model revision and frozen source. Its 13 sessions produced 33 commits and three expected refusals. These outcomes are excluded from all primary totals and timings.

The replay reproduced the original applied faults, speculative/corrected token traces and native-reference fingerprints. Sparse started at cut 7 for seven steps, then at layer 0; all-cuts used cut 14 for seven steps, then layer 0. The recorded first_divergence=7 is one-based: the seventh predicted token differs, and only the eighth step starts at layer zero. Full replay used layer 0 throughout. Three separately saved policy archives passed HPC and local CPU audits, totaling 366 recovered/reference tensor pairs, including two-step continuation. This closes the selected case's state-archive gap; it is not another independent fault sample, an additional primary performance result or new novelty evidence.

Important evidence qualifications

Exact state equality is numeric elementwise equality; signed-zero distinctions are separately byte-sensitive. The saved subset's byte-comparison outcome is reported without expanding the general detector contract. Model-file digest records bind the runtime record, but local audits do not reload remote model weights or independently regenerate tokenization and Gaussian draws.

Qwen runtime warnings about eager sliding-window attention are retained where emitted; the actual model configurations disable sliding-window execution, and native parity remains the operational gate. No physical GPU faults, binary instrumentation, malicious attacks, real failure incidence, semantic accuracy or model alignment were measured.

8. Prior work, novelty and publication judgment

Closest established mechanisms

Prior work	Overlap and the remaining distinction
LayerSkip, ACL 2024	Reuses lower-layer KV/exit activations during self-speculation and manages the accepted prefix after mismatch. Layer-cut reuse and history-valid rollback are not new general principles. Published paper
VeriCache, 2026 preprint	Full-KV verification corrects compressed-KV drafting at the first mismatch. Its deliberate compression setting differs from persistent-memory corruption, but generic exact-output verification/reuse claims overlap. Versioned paper
HCache	Reconstructs KV using stored layer-input hidden states and projection work. RECUT's all-cuts replay is not this full serving/restoration system. Versioned paper
HybridServe	Mixes activation and KV storage for recomputation/transfer tradeoffs. Fixed sparse cuts do not establish a new optimal placement policy. Versioned paper

This focused recheck examined the four primary sources and selected mechanism sections. The earlier 100-paper screen is preserved separately with its original reading-depth labels; this stage is not a fresh cover-to-cover review of 100 papers. LayerSkip coauthor Anas Mahmoud must not be confused with Prof. Abdulrahman Mahmoud.

Professor fit

Dependable AI systems, application-visible error consequences and selective protection fit Prof. Abdulrahman Mahmoud's research background. RECUT's fault/recovery evaluation is therefore a defensible supervision topic. That fit does not establish that he endorses the mechanism, accepts its thesis scope or considers it novel. [Official profile](#) and [publication list](#).

Assessment after the completed study

The engineering milestone is successful: detector-routed selective replay, permanent divergence fallback, integrity refusal and cost accounting work under the specified contract, including the 7B model. The research claim remains unproven at A/A* level. Prior work already uses intermediate-state reconstruction and history-conditioned reuse; this study has no adapted published-method baseline, optimized serving evaluation, representative failure distribution or deployment-grounded reason for the trusted checkpoint/detector boundary. The expensive clean path and token holdback are central results, not footnotes.

The defensible candidate contribution is a precisely delimited transactional KV-recovery contract plus an auditable empirical tradeoff study. Whether that constitutes a sufficiently general insight is a supervisor/reviewer question. A stronger paper would need a credible failure-and-commit setting and a demonstrably better cost/correctness frontier against strong alternatives. More runs of this same prototype alone do not establish novelty or justify an acceptance prediction.

The honest claim is a bounded detector-routed transactional KV-recovery study with measured protection costs, not "the first activation-based repair method," "provably optimal recovery," "general hardware resilience" or "A*-ready." Conference ranking or topical fit cannot substitute for a demonstrated contribution.

9. Reproducibility, completion and supervisor handoff

The measured environment was Python 3.13.9, PyTorch 2.6.0+cu124, Transformers 4.51.3, BF16, deterministic algorithms, TF32 disabled, and NVIDIA RTX 5000 Ada 32 GB GPUs. Slurm allocated one GPU, four CPUs and 48 GB host memory per task. The account allowed two simultaneous jobs; the third model waited normally. The main matrix was gated on all three debug tasks succeeding.

Model	Immutable revision
HuggingFaceTB/SmolLM2-135M	93efa2f097d58c2a74874c7e644dbc9b0cee75a2
Qwen/Qwen2.5-0.5B	060db6499f32faf8b98477b0a26969ef7d8b9987
Qwen/Qwen2.5-7B	d149729398750b98c0af14eb82c78cfe92750796

Start with `research-next/stage3/DELIVERY.md`. It gives CPU-only evidence-audit commands, the combined tests, localized reproduction rules and PDF rebuilding. The portable ZIP includes the main/debug raw records and tensor archives, frozen dependencies, corrected summaries, reporting tools and reviews. It deliberately excludes model weights, environments and authentication material.

Source freeze: `stage3/protocol_freeze.json`, written before pretrained measurements. Main data: `runs/stage3-main-v1/{smol135m,qwen05b,qwen7b}`. Audits: `audit.json` (HPC) and `audit_local.json` (local) per model. Numerical source: `notes/stage3_main_recomputed.json`. Main scheduler array: 195927; debug array: 195926. The failed initial model-fetch setup log is preserved; its successful CPU-only retry preceded measurement.

Research decision

The stage is complete when the configured experiment and its audits finish, not when its conclusion becomes favorable. The remaining publication requirements are different research questions: a credible deployment fault/commit contract, strong adapted published-method comparisons, representative optimized serving, and a general contribution beyond familiar checkpointing/reuse mechanisms. These cannot be certified by adding more repetitions to the same matrix.

If normal-case protection costs or stronger alternatives erase the useful operating region, the performance-oriented thesis claim must be revised or rejected. A negative finding would need a broadly useful explanation or bound to become a strong paper; a slow prototype alone is not enough. No threshold, fault severity or headline sample was selected after observing the final results. The explicitly separated fallback diagnostic was selected afterward for additional state validation and never enters primary performance or sample totals.

A concise explanation for your professor

I implemented a protected LLM inference-window controller that detects persistent old-prefix KV changes and reuses saved lower-layer work only while corrected token history agrees. I evaluated it on three pinned models through 7B, with K/V faults, integrity challenges and full-state checks. Late-layer affected windows were about 1.38-1.53x faster than our equally protected full-replay baseline, but clean protection overhead reached about 50% versus native inference in the 7B long-shape case, and output buffering delayed availability. The next research decision is whether a realistic reliability setting and a more general contribution can justify those costs; the current artifact alone is not an A/A*-ready paper.

Delivery boundaries

Source, raw evidence, preserved failures, scheduler logs, audits, result recomputation, this report and a hash-verified archive are provided. Model weights, environments and credentials are not bundled. Earlier stages and other thesis projects are preserved. No paper was submitted and no GitHub repository or production cloud cluster was modified as part of this stage.

All implementation, reviews and writing are AI-assisted. Human authors must verify claims, attribution and applicable AI-use policies before any submission. This document is supervisor-review material, not a substitute for that review.